

ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ

ΑΝΑΦΟΡΑ ΠΑΡΑΔΟΣΗΣ ΕΡΓΑΣΙΑΣ 2

Στοιχεία μελών ομάδας

Ονοματεπώνυμο	Αριθμός Μητρώου
ΜΟΝΟΧΟΛΙΑΣ ΙΩΑΝΝΗΣ	3230128
ΜΠΑΝΟΣ ΒΑΣΙΛΕΙΟΣ	3230132

ΜΕΡΟΣ Α(Υλοποίηση ΑΤΔ)

Στο μέρος Α της εργασίας υλοποιούνται οι ΑΤΔ που θα χρειαστούν για τα επόμενα στάδια της εργασίας. Συγκεκριμένα, δημιουργήθηκαν, βάσει της εκφώνησης, οι κάτωθι ΑΤΔ:

Processor
private int id private List processedJobs
public int getTotalProcessingTime() public void addJob(Job job) int getId() public int compareTo(Processor B) public String toString()

Σημείωση: Η λίστα processedJobs της κλάσης Processor υλοποιήθηκε με συνδεδεμένη λίστα, στα πρότυπα αυτής του εργαστηρίου, με κόμβους τύπου Node<Job>.

Job
int id int time
public Job() public Job(int id,int time) public String toString() public int compareTo(Job other)

Σημείωση: Η μέθοδος public int compareTo(Job other) υλοποιήθηκε για σύγκριση του χρόνου των διεργασιών για τις ανάγκες του μέρους Γ .

MaxPQ
private Processor[] heap private int size private final int DEFAULT_CAPACITY = 7
public MaxPQ() public boolean isEmpty() public int size() public Processor max() public void maxAddJob(Job j) public void insert(Processor x) public Processor getMax() private void sink(int i) private void resize() private void swim(int i)

Σημείωση: Η μόνη μέθοδος που προστέθηκε σε σχέση με τον κώδικα του εργαστηρίου είναι η public void maxAddJob(Job j), η οποία προσθέτει μία διεργασία στον επεξεργαστή με τη μεγαλύτερη προτεραιότητα και επαναφέρει την ιδιότητα του σωρού, σε περίπτωση που αυτή διαταράχθηκε, μέσω της προσθήκης της διεργασίας.

ΜΕΡΟΣ Β(Αλγόριθμος 1)

Για την υλοποίηση του **Αλγορίθμου 1 (Greedy)** έγινε χρήση της ουράς προτεραιότητας(MaxPQ) που δημιουργήθηκε κατά το πρώτο μέρος της εργασίας και υλοποιεί τη διεπαφή PQInterface. Η εν λόγω ουρά αποθηκεύει αντικείμενα τύπου Processor, χρησιμοποιώντας μεγιστοστρεφή σωρό.

Αρχικά, το πρόγραμμα δέχεται ως είσοδο από τη γραμμή εντολών το όνομα του αρχείου .txt προς ανάγνωση, η οποία γίνεται με τη χρήση ενός αντικειμένου της κλάσης **BufferedReader**, ενώ αρχικοποιεί και την ουρά προτεραιότητας που πρόκειται να αποθηκεύσει τους προς ανάθεση διεργασίες επεξεργαστές.

Στο Β Μέρος γίνεται επίσης χρήση τεσσάρων μεθόδων για να είναι ευκρινέστερα τα βήματα της υλοποίησής του. Αυτές είναι οι εξής:

- **static void PQInitialization(BufferedReader reader,MaxPQ processorsPQ) throws IOException**

Δέχεται ως ορίσματα το αντικείμενο reader(για να αναγνώσει το αρχείο) και την ουρά προτεραιότητας. Αρχικοποιεί την ουρά προτεραιότητας με τόσους επεξεργαστές, όσους διάβασε από το αρχείο.

- **static int JobsNumCheck(BufferedReader reader,String filename) throws IOException**

Η Συγκεκριμένη μέθοδος δέχεται ως ορίσματα το αντικείμενο reader(για να αναγνώσει το αρχείο) και το όνομα του αρχείου(για να το εμφανίσει σε περίπτωση εντοπισμού λάθους).Στη συνέχεια διαβάζει γραμμή προς γραμμή το αρχείο για να διαπιστώσει αν ο αριθμός των διεργασιών που διαβάστηκε αντιστοιχεί σε αυτόν που πραγματικά υπάρχει στο αρχείο. Η συμπεριφορά της μεθόδου στην περίπτωση των σωστών ή λάθος δεδομένων φαίνεται στον παρακάτω πίνακα.

Περίπτώσεις	Συμπεριφορά μεθόδου
1.Αναντιστοιχία αριθμού διεργασιών που αναμένεται σε σχέση με αυτές που πραγματικά υπάρχουν στο αρχείο	Εκτύπωση μηνύματος λάθους στο αρχείο και επιστροφή της τιμής -1 στη main ώστε να τερματίσει μετά το σχετικό έλεγχο
2.Κανένα πρόβλημα αναντιστοιχίας	Επιστροφή του αριθμού των διεργασιών και συνέχιση της main

- **static Job[] StoreJobs(BufferedReader reader,MaxPQ processorsPQ,int jobsCounter) throws IOException**

Η μέθοδος αυτή δέχεται ως ορίσματα το αντικείμενο reader(για να αναγνώσει το αρχείο) και τον συνολικό αριθμό των διεργασιών. Η λειτουργίας της είναι απλά η δημιουργία αντικειμένων τύπου Job με βάση τα δεδομένα που της παρέχει το αρχείο και έπειτα η αποθήκευσή τους στον πίνακα Jobs, ο οποίος επιστρέφεται μετά την εκτέλεση της μεθόδου.

- **static int FinalMessageAndCalculations(MaxPQ processorsPQ,int jobsCounter)**

Δέχεται ως ορίσματα την ουρά προτεραιότητας και τον αριθμό των διεργασιών .Αδειάζει σε ένα while-loop την ουρά προτεραιότητας και εμφανίζει όλα τα στοιχεία της (αν είναι λιγότερα από 50).Τέλος, επιστρέφει, το makespan που υπολόγισε.

Η χρήση της ουράς προτεραιότητας γίνεται για να φορτωθεί (καθώς διατρέχουμε τον πίνακα των διεργασιών που διαβάστηκαν) η παρούσα διεργασία στον επεξεργαστή που έχει εκείνη τη στιγμή τη μεγαλύτερη προτεραιότητα, δηλαδή έχει το λιγότερο φόρτο. Αυτό επιτυγχάνεται με τη χρήση της μεθόδου **maxAddJob** της κλάσης MaxPQ ,η οποία λειτουργεί ως εξής:

Προσθέτει τη διεργασία στον επεξεργαστή με τη μεγαλύτερη προτεραιότητα και αφού ενημερωθεί ο συνολικός του φόρτος κάνει sink, γιατί μπορεί μέσω της προσθήκης της διεργασίας στον επεξεργαστή να διαταράχθηκε η ιδιότητα του σωρού.

Κάποιες λεπτομέρειες για τον κώδικα του μέρους Β:

- Το αρχείο βρίσκεται στο φάκελο data και το όνομα του αρχείου δίνεται ως όρισμα στο command line ως εξής: `java Greedy data/B_input.txt`
- Για την ανάγνωση του αρχείου χρησιμοποιείται η κλάση της Java **BufferedReader**
- Το πρόγραμμα συλλαμβάνει τις εξαιρέσεις, αν αυτές εμφανιστούν.
- Υπάρχουν αναλυτικά σχόλια σε όλη την έκταση του κώδικα με λεπτομέρειες για τη χρήση συγκεκριμένων μεταβλητών , συνθηκών , βρόχων

ΜΕΡΟΣ Γ(Αλγόριθμος Ταξινόμησης Heapsort)

Εφόσον ο ΑΜ και των 2 μελών της ομάδας λήγει σε άρτιο αριθμό, ο αλγόριθμος που υλοποιήθηκε για το Μέρος Γ της εργασίας είναι η ταξινόμηση Heapsort με χρήση πίνακα. Αυτός υλοποιείται στην κλάση Sort μέσω της στατικής μεθόδου HeapSort, που δέχεται ως όρισμα τον πίνακα που πρόκειται να ταξινομήσει και εφαρμόζει τον αλγόριθμο Heapsort πάνω στον πίνακα.

ΜΕΡΟΣ Δ(Πειραματική αξιολόγηση αλγορίθμων 1 και 2)

Για την υλοποίηση του μέρους Δ δημιουργήθηκαν 30 συνολικά αρχεία, 10 για κάθε μία εκ των 7 τιμών {100,150,250,350,450,500,600} , που δηλώνουν τον αριθμό των διεργασιών που περιέχει το κάθε αρχείο.

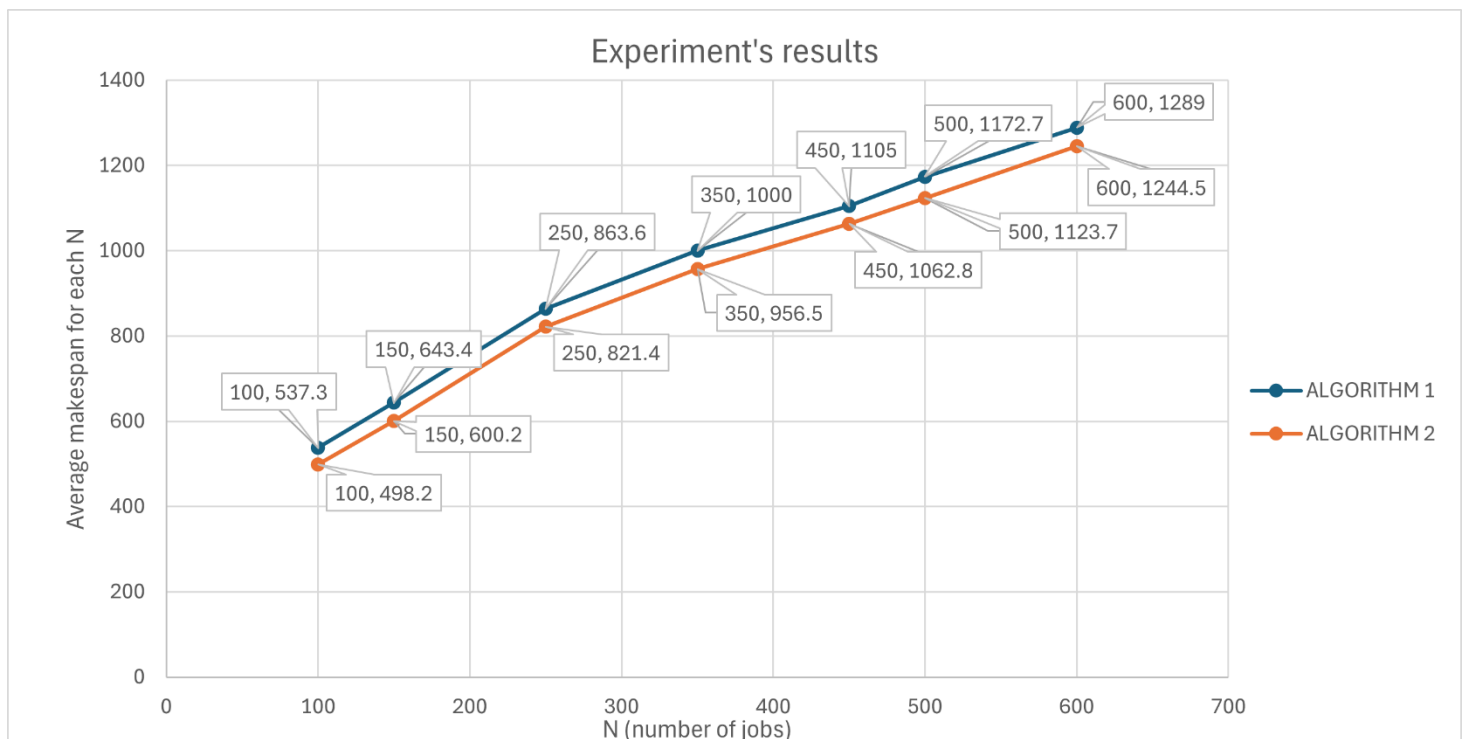
Δημιουργία των αρχείων

Τα αρχεία δημιουργήθηκαν σε ένα for-loop, το οποίο κάνει 10 επαναλήψεις, για κάθε τιμή του $n=100,150,250,350,450,500,600$ οπότε συνολικά δημιουργούνται 30 αρχεία .txt. Έτσι, ένα αντικείμενο της κλάσης FileWriter γράφει στο κάθε αρχείο η διεργασία, οι οποίες θα κατανεμηθούν σε $\sqrt{n}$ επεξεργαστές. Επίσης, ο χρόνος που παίρνει κάθε διεργασία είναι τυχαίος και με μέγιστη τιμή το 100, με τη βοήθεια ενός αντικειμένου της κλάσης Random.

Υπολογισμοί του προγράμματος

Το πρόγραμμα ακολουθεί την ίδια λογική ως προς τη ροή εκτέλεσης και την κλήση μεθόδων για την υλοποίηση των δύο αλγορίθμων, με μόνη διαφορά ότι για τον αλγόριθμο 2 οι διεργασίες ταξινομούνται(με τον αλγόριθμο HeapSort) πριν δρομολογηθούν στους επεξεργαστές. Συγκεκριμένα, για κάθε τιμή του n γίνεται ανάγνωση των 10 αρχείων του συγκεκριμένου n , ώστε να εφαρμοστούν οι αλγόριθμοι 1 και 2 σε κάθε ένα αρχείο από αυτά και να καταγραφεί το makespan του κάθε αρχείου. Στο τέλος, λαμβάνεται ο μέσος όρος των Makespans για το συγκεκριμένο n και τυπώνονται στο χρήστη τα αποτελέσματα των επιδόσεων των 2 αλγορίθμων για κάθε τιμή του n .

Στο διάγραμμα της επόμενης σελίδας φαίνονται οπτικοποιημένα τα αποτελέσματα του πειράματος για κάθε τιμή του n . Για την κατασκευή των διαγραμμάτων χρησιμοποιήθηκε το Excel και οι τιμές του n είναι περισσότερες από αυτές που ζητούνται στην εκφώνηση της εργασίας.



Διάγραμμα Excel της πειραματικής αξιολόγησης των δύο αλγορίθμων

Συμπεράσματα πειραματικής αξιολόγησης των 2 αλγορίθμων

Είναι εμφανές από το διάγραμμα ότι η μέση τιμή των makespans για κάθε τιμή του N είναι μικρότερη για τον αλγόριθμο 2. Συγκεκριμένα, παρατηρείται ότι για όλες τις τιμές του N το makespan του αλγορίθμου 2 είναι μικρότερο κατά 40-55 μονάδες σε σχέση με αυτό του αλγορίθμου 1, γεγονός που επαληθεύει την παρατήρηση του μέρους Γ ότι η ταξινόμηση κατά φθίνουσα σειρά των διεργασιών βελτιώνει/μειώνει το makespan.

Κάποιες λεπτομέρειες για τον κώδικα του μέρους Δ:

- Ο κώδικας ανάγνωσης των αρχείων βρίσκεται εντός σχολίων. Ο λόγος που συμβαίνει αυτό είναι να μη δημιουργούνται νέα αρχεία κάθε φορά που τρέχει το πρόγραμμα.
- Για ανάγνωση και εγγραφή χρησιμοποιούνται οι κλάσεις της Java **BufferedReader** και **FileWriter**.
- Όλα τα αρχεία που δημιουργήθηκαν και διαβάστηκαν είναι αποθηκευμένα στον φάκελο data.

Όπως και στο μέρος Β:

- Το πρόγραμμα συλλαμβάνει τις εξαιρέσεις, αν αυτές εμφανιστούν.
- Υπάρχουν αναλυτικά σχόλια σε όλη την έκταση του κώδικα με λεπτομέρειες για τη χρήση συγκεκριμένων μεταβλητών, συνθηκών, βρόχων.